

Threads

Samuel Francisco Ferrigo

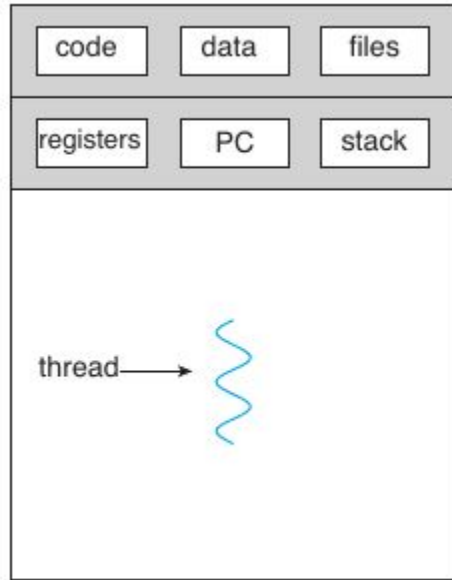
Conceito

Cada processo tem um espaço de endereçamento e uma thread de controle

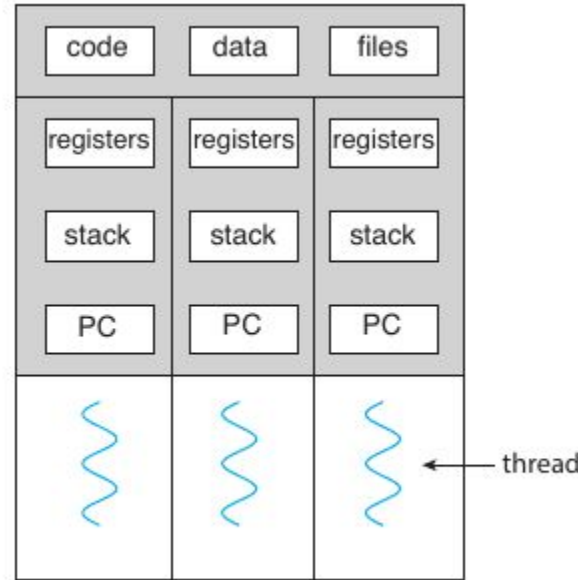
Entretanto há situações onde um processo pode ter várias threads de controle

Uma thread compartilha o código, os dados e recursos do SO com outras threads

Por isso, uma thread é considerada um processo “leve”

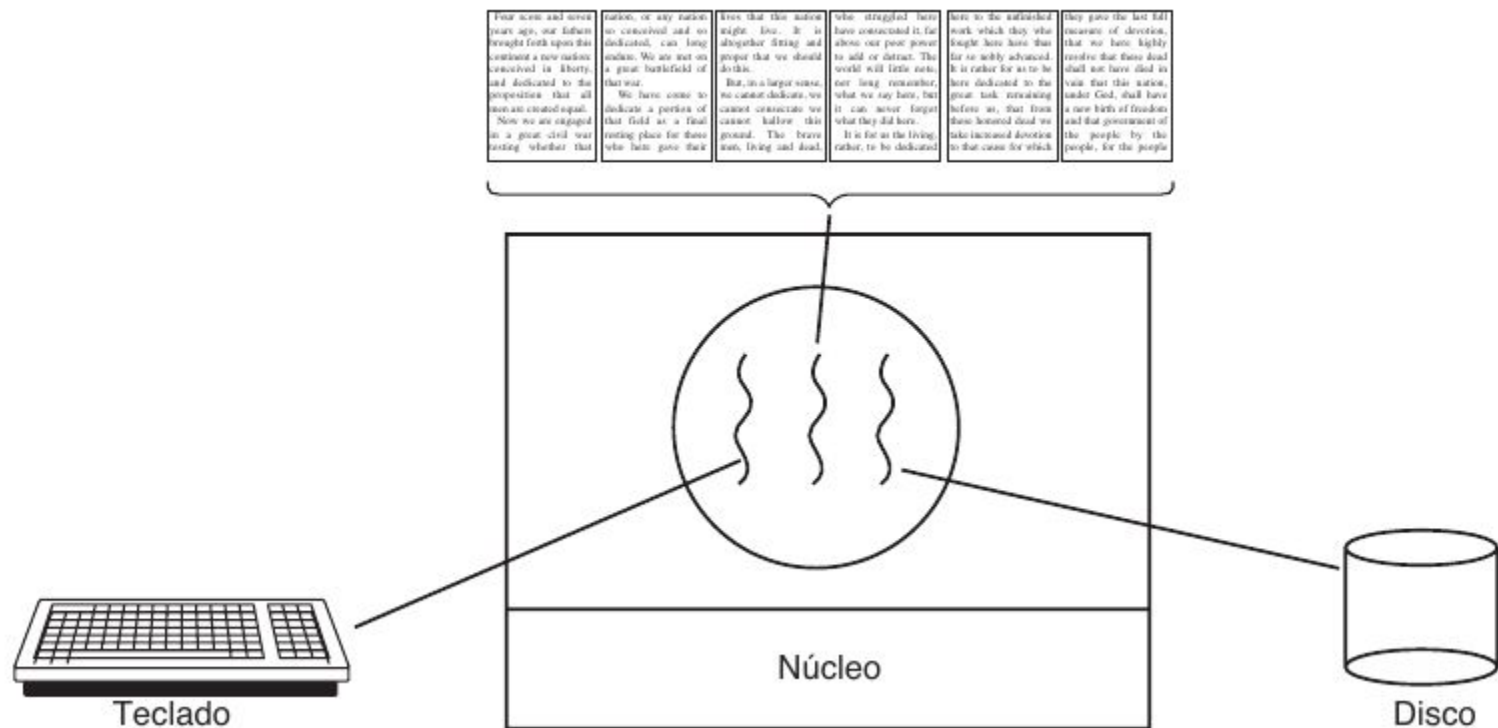


single-threaded process



multithreaded process

Estrutura de um processo, e a localização das threads dentro dela. À direita, um processo multithreading



Um exemplo de aplicação de multithread, num processador de texto

Processos vs threads

Um processo é uma instância do programa em execução em um sistema operacional, enquanto uma thread é uma unidade de processamento dentro de um processo.

Os processos têm sua própria área de memória separada e recursos de sistema, como arquivos abertos e conexões de rede.

Os processos são isolados uns dos outros e comunicam-se por meio de mecanismos de comunicação interprocessual (IPC), como pipes, sockets e sinais.

Cada processo tem pelo menos uma thread, que é chamada de thread principal.

Processos vs threads

As threads, por outro lado, compartilham a mesma área de memória e recursos de sistema dentro do processo

Portanto, elas são menores e mais leves do que os processos e são usadas para executar tarefas paralelas dentro de um único processo

Elas são gerenciadas pelo SO e pelo próprio programa, que deve garantir a sincronização adequada para evitar problemas de concorrência

São normalmente mais rápidas que os processos

Porém, são mais suscetíveis a problemas de sincronização e concorrência

Itens por processo	Itens por thread
Espaço de endereçamento	Contador de programa
Variáveis globais	Registradores
Arquivos abertos	Pilha
Processos filhos	Estado
Alarmes pendentes	
Sinais e tratadores de sinais	
Informação de contabilidade	

As threads requerem a guarda de muito menos informações. À esquerda, alguns itens compartilhados por todas as threads. À direita, itens específicos de cada thread

Benefícios do uso de multithreading

Responsividade

- Permite a execução de um programa mesmo se parte dele está bloqueado ou realizando uma operação lenta
- Útil em interfaces de usuário, na captura de um clique do mouse, por exemplo

Compartilhamento e economia de recursos

- Em especial de memória

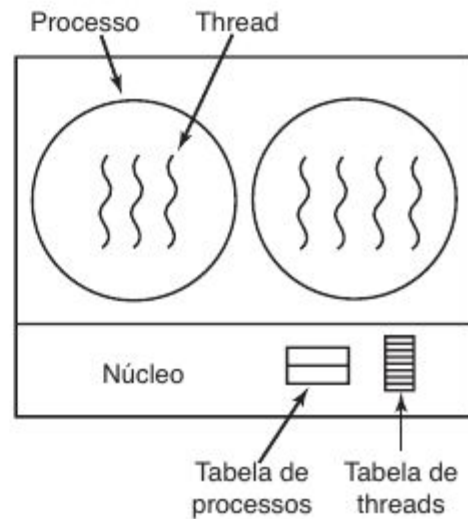
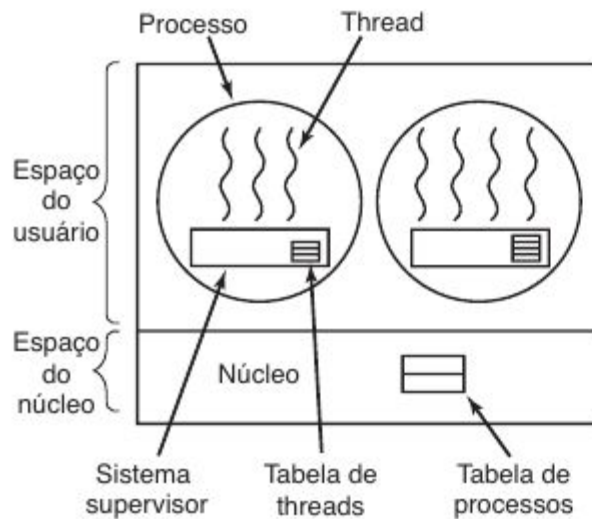
Escalabilidade

- Permite a execução paralela em múltiplos processadores

Implementação das threads

As threads podem ser implementadas no

- Espaço do usuário
 - Implantado por biblioteca (como pthreads)
 - O kernel não conhece nada a respeito das threads
 - Pode ser implantado em qualquer SO
 - Cada processo tem sua tabela de threads privada
 - Permite o escalonamento customizado das threads
- Kernel
 - Responsável pelo controle das threads, através de tabela própria
 - Mais lenta que as threads de usuário, pois o chaveamento entre threads ocorre por chamadas de sistema
 - Requer menos esforço do programador



Threads implementadas no espaço do usuário (esquerda) vs implementadas no núcleo (direita)

POSIX Threads

Padrão definido pela IEEE nos anos 1980 para a implantação de threads no espaço do usuário

Funções definidas na biblioteca <pthread.h>

Principais funções

Iniciar uma thread

```
int pthread_create (pthread_t *thread, pthread_attr_t *attr, void *(*start_routine) (void *), void *arg);
```

- *thread - ponteiro para a thread
- *attr - atributos da thread, como execução acoplada ou desacoplada de demais threads. Normalmente informado NULL.
- *(*start_routine) - ponteiro para a função que será executada pela thread
- *arg - ponteiro para os parâmetros a serem executados pela thread

Principais funções

Finalizar uma thread

```
void pthread_exit (void *retval);
```

- *retval é o valor de retorno que pode ser usado por outras threads.
Normalmente NULL

Principais funções

Sincroniza as threads com a thread principal (programa principal)

```
int pthread_join (pthread_t thread, void **thread_return);
```

- *thread - ponteiro para a thread
- **thread_return - ponteiro com o valor retornado pela função executada pela thread. Normalmente NULL.

Compilando

```
gcc -pthread arquivo_fonte.c -o arquivo_executavel
```

Kthread

API do kernel Linux usada para implementação de Threads diretamente no núcleo

A implementação é mais custosa que em PThreads

Bastante utilizada para desenvolvimento de drivers